This paper is part of the refereed proceedings of the 2026 INFORMS Optimization Society Conference, Atlanta, GA, USA, March 20–22, 2026, https://ios2026.isye.gatech.edu/refereed-proceedings.

A dive-and-fix approach for large-scale pooling problems

Ke-Ni Xiang[a], Zhi-Wei Wei[b], Xin Sun[b],Yan-Ru Wang[a], Yan-Ming Cao[b], Wei-Kun Chen[a], Yu-Hong Dai[c]

[a] School of Mathematics and Statistics, Beijing Institute of Technology, {kenixiang ,chenweikun,wangyanru}@bit.edu.cn

[b] Petrochina Planning and Engineering Institute and CNPC Laboratory of Oil & Gas Business Chain Optimization, {weizhiwei,sunxin6,caoyanming}@petrochina.com.cn

[c] Academy of Mathematics and Systems Science, Chinese Academy of Sciences, dyh@lsec.cc.ac.cn

Abstract. The pooling problem is an NP-hard nonconvex bilinear programming problem that has many industrial applications, including petrochemical refining, wastewater treatment, natural gas transportation, open-pit mining, and animal feed problems. In this paper, we develop a dive-and-fix algorithm which starts with the all zeros trivial solution and then iteratively augments the current solution by involving a new terminal node until all terminal nodes are considered. Two key advantages of the proposed dive-and-fix algorithm that make it particularly suitable for solving large-scale pooling problems are as follows. First, it only requires solving linear programming subproblems, each of which considers a single terminal node. Second, it guarantees to return a feasible solution (usually with high solution quality). As a byproduct of analysis, we prove that checking whether the optimal value of a pooling problem is zero can be accomplished in polynomial time, which closes an open question in the literature. Finally, computational results demonstrate the effectiveness and efficiency of the proposed dive-and-fix algorithm over the direct use of an off-the-shelf global solver and a state-of-the-art greedy construction heuristic algorithm in the literature.

Key words: Pooling problem, augmentation problem, dive-and-fix

1. Introduction

The pooling problem is a classic network flow problem on a directed graph with three layers of nodes: source nodes, pool nodes, and terminal nodes. This problem involves directing flow from source nodes to terminal nodes, potentially routing it through intermediate pool nodes. The flow originating from the source nodes is characterized by known attribute qualities. At the pool and terminal nodes, incoming flows are blended, with the attribute qualities mixing linearly, that is, the attribute qualities at any given node are combined in proportion to the incoming flows. The objective of the pooling problem is to minimize the total weight (or cost) associated with routing these flows while satisfying capacity constraints of the nodes and arcs, as well as the attribute quality requirements at the terminal nodes. The problem has a wide variety of applications, including petrochemical refining (Haverly 1978, Baker and Lasdon 1985), wastewater treatment (Galan and Grossmann 1998, Misener and Floudas 2010), natural gas transportation (Rømo et al. 2009, Ríos-Mercado and Borraz-Sánchez 2015), open-pit mining (Blom et al. 2014, Boland et al. 2017), and animal feed problems (Grothey and McKinnon 2023).

Due to its wide range of applications, various algorithms have been proposed in the literature. One primary category of algorithms is based on the branch-and-bound (B&B) framework. These include global algorithms (Tawarmalani and Sahinidis 2002, Misener et al. 2011, Luedtke et al. 2020) based on various bilinear programming formulations (Haverly 1978, Ben-Tal et al. 1994, Audet et al. 2004, Alfaki and Haugland 2013a,b, Boland et al. 2016, Grothey and McKinnon 2023) that guarantee to return an optimal solution, and discretization methods (Pham et al. 2009, Dey and Gupte 2015, Castro 2023), which first construct mixed-integer linear programming (MILP) restrictions and then solve the resultant problem using off-the-shelf MILP solvers. While B&B

algorithms are effective in finding high-quality feasible or optimal solutions, due to the exponential complexity, they typically exhibit low computational efficiency.

To bypass the difficulty, various local heuristic algorithms have been proposed in the literature, including successive linear programming type algorithms (Haverly 1978, Lasdon and Joffe 1990), variable neighborhood search (Audet et al. 2004), greedy construction heuristic (Alfaki and Haugland 2014), and generalized Benders decomposition heuristic search (Floudas and Aggarwal 1990); see Misener and Floudas (2009), Gupte et al. (2017) for a comprehensive review of algorithms of solving pooling problems. Unfortunately, most of these heuristic algorithms do not provide a performance guarantee; in particular, they may even fail to return a feasible solution.

The motivation of this paper is to propose an algorithm that guarantees to find a feasible solution for the pooling problem (likely with a high solution quality) while maintaining high computational efficiency. In particular, we revisit the terminal proportion (TP) formulation of Alfaki and Haugland (2013b), Boland et al. (2016) involving the routing and quality proportion variables, and transform it into a formulation that only involves the routing variables. Based on the new formulation, we investigate a class of restriction problems in which only a subset of terminal nodes is considered and discuss how to augment an existing feasible solution, by incorporating a new terminal node, to obtain a better feasible solution for the pooling problem. The latter can be achieved by solving a linear programming (LP) problem. As a byproduct of analysis, we prove that checking whether the optimal value of a pooling problem is zero can be accomplished in polynomial time, which closes the open question posed by Gupte et al. (2017). We develop a dive-and-fix heuristic algorithm that starts with the all zeros trivial solution and then iteratively augments the current solution by adding a new terminal node that provides the greatest objective improvement (among all unconsidered terminal nodes) until all terminal nodes are included. Two key advantages of the proposed algorithm that make it particularly suitable for solving large-scale pooling problems are as follows: (i) it only requires solving LP subproblems, each of which considers a single terminal node; (ii) it guarantees to return a feasible solution for the pooling problem (usually with a high solution quality). Through computational experiments, we show the effectiveness and efficiency of the proposed dive-and-fix algorithm over the direct use of the off-the-shelf global solver Baron and the state-of-the-art greedy construction heuristic algorithm of Alfaki and Haugland (2014).

The rest of the paper is organized as follows. Section 2 introduces the problem formulations. Section 3 develops the dive-and-fix heuristic algorithm. Section 4 reports the computational results.

2. Problem formulations

Consider a directed graph $G = (N, A)$, where N and A are the sets of nodes and arcs, respectively. The nodes in N are categorized into three mutually exclusive types: source nodes in S, pool nodes in P, and terminal nodes in T. The set of arcs A is a subset of $(S \times P) \cup (S \times T) \cup (P \times P) \cup (P \times T)$. In this paper, we address the generalized pooling problem (Audet et al. 2004, Meyer and Floudas 2006), which allows for arcs between pool nodes.

Each node $j \in N$ is assigned a capacity u_j. In particular, u_s denotes the total supply of raw materials at source $s \in S$, u_p stands for the processing capacity at pool $p \in P$, and u_t indicates the maximum demand for products at terminal $t \in T$. For each arc $(i, j) \in A$, the parameter u_{ij} specifies the upper bound for the amount of flow along the arc, while c_{ij} indicates the weight (or cost) of a unit flow sent from node i to node j.

Let K be the set of attributes. The quality of attribute $k \in K$ at source $s \in S$ is denoted as λ_{sk}. In this work, we assume that the flow is blended linearly (Gupte et al. 2017), that is, the quality of attribute $k \in K$ at a node $j \in P \cup T$ is a weighted average of those of the incoming flows. At each

terminal $t \in T$, the flow has minimum and maximum quality thresholds for each attribute $k \in K$, denoted by $\lambda_{tk}^{\min}$ and $\lambda_{tk}^{\max}$.

The pooling problem aims to determine a flow routing that minimizes the total weight (or cost), subject to the capacity constraints on nodes and arcs and the attribute quality constraints at the terminal nodes. Various mathematical formulations for the pooling problem have been proposed in the literature; see Haverly (1978), Ben-Tal et al. (1994), Tawarmalani and Sahinidis (2002), Audet et al. (2004), Alfaki and Haugland (2013a,b), Boland et al. (2016), Grothey and McKinnon (2023). In the following, we will focus on the TP formulation (Alfaki and Haugland 2013b, Boland et al. 2016), as it is highly relevant to our work.

2.1. Terminal proportion formulation for the pooling problem

To present the TP formulation, the following notations are needed. Let $N_j^- := \{i \in N : (i,j) \in A\}$ and $N_j^+ := \{i \in N : (j,i) \in A\}$ denote the sets of in-neighbors and out-neighbors of node $j \in N$, respectively, $T_p \subseteq T$ be the subset of terminals reachable from a pool $p \in P$, and $S_t \subseteq S$ and $P_t \subseteq P$ be the sets of sources and pools that have a path to terminal t. We use variable y_{ij} to denote the flow on arc $(i,j) \in A$, variable x_{ip}^t to represent the flow on arc $(i,p) \in A$ (where $p \in P$ and $i \in N_p^-$) that is destined to terminal $t \in T_p$, and variable q_p^t to denote the proportion of the total flow at pool $p \in P$ that is destined to terminal $t \in T_p$. Then, the TP formulation (Alfaki and Haugland 2013b, Boland et al. 2016) for the pooling problem can be written as:

$$\min_{x,y,q} \left\{ \sum_{t \in T} \left(\sum_{p \in P_t} \sum_{i \in N_p^-} c_{ip} x_{ip}^t + \sum_{i \in N_t^-} c_{it} y_{it} \right) : (1a)\text{–}(1i) \right\}, \tag{TP}$$

where

$$\sum_{t \in T} \left(\sum_{p \in P_t \cap N_i^+} x_{ip}^t \right) + \sum_{t \in N_i^+ \cap T} y_{it} \le u_i, \ \forall\, i \in S \cup P, \tag{1a}$$

$$\sum_{i \in N_t^-} y_{it} \le u_t, \ \forall\, t \in T, \tag{1b}$$

$$\sum_{t \in T_p} x_{ip}^t \le u_{ip}, \ \forall\, p \in P,\ i \in N_p^-, \qquad y_{it} \le u_{it}, \ \forall\, t \in T,\ i \in N_t^-, \tag{1c}$$

$$\sum_{i \in N_p^-} x_{ip}^t = \sum_{p' \in P_t \cap N_p^+} x_{pp'}^t + \chi_{pt} y_{pt}, \ \forall\, t \in T,\ p \in P_t, \tag{1d}$$

$$\lambda_{tk}^{\min} \sum_{i \in N_t^-} y_{it} \le \sum_{s \in S_t} \sum_{p \in P_t \cap N_s^+} \lambda_{sk} x_{sp}^t + \sum_{s \in S_t \cap N_t^-} \lambda_{sk} y_{st} \le \lambda_{tk}^{\max} \sum_{i \in N_t^-} y_{it}, \ \forall\, t \in T,\ k \in K, \tag{1e}$$

$$x_{ip}^t = q_p^t y_{ip}, \ \forall\, t \in T,\ p \in P_t,\ i \in N_p^-, \tag{1f}$$

$$\sum_{t \in T_p} q_p^t = 1, \ \forall\, p \in P, \tag{1g}$$

$$x_{ip}^t \in \mathbb{R}_+, \ \forall\, t \in T,\ p \in P_t,\ i \in N_p^-,\ y_{ij} \in \mathbb{R}_+, \ \forall\, j \in P \cup T,\ i \in N_j^-, \tag{1h}$$

$$q_p^t \in [0,1], \ \forall\, t \in T,\ p \in P_t. \tag{1i}$$

The objective is to minimize the total weight associated with routing the flows. Constraints (1a)–(1c) enforce the capacity restrictions on the nodes and arcs. Constraints (1d) ensure the flow conservation at each pool p, where $\chi_{pt} = 1$ if $(p,t) \in A$ and $\chi_{pt} = 0$ otherwise. Constraints (1e) require the quality attributes at each terminal to be within the required thresholds. Constraints (1f) ensure that

the flow on arc (i, p) times the proportion of the flow at pool p destined to terminal t is equal to the flow on arc $(i, p) \in A$ destined to terminal $t \in T_p$, and constraints (1g) enforce that at each pool, the sum of the proportions of the flow destined to the terminals must be equal to one.

2.2. Projecting the proportion variables out from the formulation

Problem (TP) is a nonconvex nonlinear programming (NLP) problem that involves routing variables x and y and proportion variables q. The following proposition shows that proportion variables q can be equivalently projected out from formulation (TP).

PROPOSITION 1. *If (x, y, q) satisfies the system (1f)–(1i), then (x, y) satisfies the system (1h) and*

$$x^t_{i_1 p} y_{i_2 p} = x^t_{i_2 p} y_{i_1 p}, \ \forall\, t \in T,\ p \in P_t,\ i_1, i_2 \in N^-_p \text{ with } i_1 \neq i_2, \tag{2}$$

$$y_{ip} = \sum_{t \in T_p} x^t_{ip}, \ \forall\, p \in P,\ i \in N^-_p. \tag{3}$$

Conversely, if (x, y) satisfies the system (1h), (2), and (3), then there exists a vector q for which the system (1f)–(1i) holds at (x, y, q).

Proof. Suppose that (x, y, q) satisfies the system (1f)–(1i). From (1f), we have $x^t_{i_1 p} y_{i_2 p} = q^t_p y_{i_1 p} y_{i_2 p} = x^t_{i_2 p} y_{i_1 p}$; therefore (2) holds at (x, y). Summing (1f) over $t \in T_p$ and using (1g) yields $\sum_{t \in T_p} x^t_{ip} = \sum_{t \in T_p} q^t_p y_{ip} = y_{ip}$, and therefore (3) also holds at (x, y). Conversely, suppose that (x, y) satisfies the system (1h), (2), and (3). Let q be defined as follows:

$$q^t_p = \begin{cases} x^t_{i_0 p} / y_{i_0 p}, & \text{if } y_{i_0 p} > 0 \text{ for some } i_0 \in N^-_p; \\ 1/|T_p|, & \text{otherwise,} \end{cases} \quad \forall\, t \in T,\ p \in P_t.$$

By direct verification, (1f)–(1i) hold at (x, y, q). □

Proposition 1 implies that we can replace (1f)–(1g) and (1i) by (2)–(3) in formulation (TP), obtaining an equivalent formulation:

$$\min_{x, y} \left\{ \sum_{t \in T} \left(\sum_{p \in P_t} \sum_{i \in N^-_p} c_{ip} x^t_{ip} + \sum_{i \in N^-_t} c_{it} y_{it} \right) : \text{(1a)–(1e)},\ \text{(1h)},\ \text{(2)–(3)} \right\}, \tag{TP'}$$

which involves only the (x, y) variables. This transformation plays a crucial role in the algorithmic design of the coming dive-and-fix approach.

3. A dive-and-fix approach for the pooling problem

In this section, we first consider restriction problems of problem (TP') where only a subset of the terminals is considered and analyze their relation to the original problem (TP'). Then, we show how to augment an existing feasible solution, for which only a subset of terminals is considered, by additionally considering another terminal. Finally, we present the dive-and-fix approach for solving problem (TP').

3.1. Restriction problem

We first present a restriction problem of formulation (TP') in which only the terminals in a subset $C \subseteq T$ are considered. In particular, letting $S_C := \cup_{t \in C} S_t$ and $P_C := \cup_{t \in C} P_t$ denote the subsets of sources and pools, respectively, that have a path to at least one terminal in C, and

$x^C := \{x^t_{ip}\}_{t\in C,\, p\in P_t,\, i\in N_p^-}$ and $y^C := \{y_{ij}\}_{j\in P_C\cup C,\, i\in N_j^-}$ denote the routing variables associated with terminals in C, then the restriction of problem (TP') can be presented as

$$\min_{x^C,\, y^C} \left\{ \sum_{t\in C} \left(\sum_{p\in P_t} \sum_{i\in N_p^-} c_{ip} x^t_{ip} + \sum_{i\in N_t^-} c_{it} y_{it} \right) : (4a)\text{–}(4h) \right\}, \tag{TP(C)}$$

where

$$\sum_{t\in C} \left(\sum_{p\in P_t\cap N_i^+} x^t_{ip} \right) + \sum_{t\in N_i^+\cap C} y_{it} \le u_i,\ \forall\, i \in S_C \cup P_C, \tag{4a}$$

$$\sum_{i\in N_t^-} y_{it} \le u_t,\ \forall\, t \in C, \tag{4b}$$

$$\sum_{t\in T_p\cap C} x^t_{ip} \le u_{ip},\ \forall\, p \in P_C,\ i \in N_p^-, \qquad y_{it} \le u_{it},\ \forall\, t \in C,\ i \in N_t^-, \tag{4c}$$

$$\sum_{i\in N_p^-} x^t_{ip} = \sum_{p'\in P_t\cap N_p^+} x^t_{pp'} + \chi_{pt} y_{pt},\ \forall\, t \in C,\ p \in P_t, \tag{4d}$$

$$\lambda^{\min}_{tk} \sum_{i\in N_t^-} y_{it} \le \sum_{s\in S_t} \sum_{p\in P_t\cap N_s^+} \lambda_{sk} x^t_{sp} + \sum_{s\in S_t\cap N_t^-} \lambda_{sk} y_{st} \le \lambda^{\max}_{tk} \sum_{i\in N_t^-} y_{it},\ \forall\, t \in C,\ k \in K, \tag{4e}$$

$$x^t_{i_1 p} y_{i_2 p} = x^t_{i_2 p} y_{i_1 p},\ \forall\, t \in C,\ p \in P_t,\ i_1, i_2 \in N_p^- \text{ with } i_1 \ne i_2, \tag{4f}$$

$$y_{ip} = \sum_{t\in T_p\cap C} x^t_{ip},\ \forall\, p \in P_C,\ i \in N_p^-, \tag{4g}$$

$$x^t_{ip} \in \mathbb{R}_+,\ \forall\, t \in C,\ p \in P_t,\ i \in N_p^-,\ y_{ij} \in \mathbb{R}_+,\ \forall\, j \in P_C \cup C,\ i \in N_j^-. \tag{4h}$$

Mathematically, problem (TP(C)) can be obtained by setting

$$y_{ij} = 0,\ \forall\, j \in (P\cup T)\backslash(P_C \cup C),\ i \in N_t^-,\ \ x^t_{ip} = 0,\ \forall\, t \in T\backslash C,\ p \in P_t,\ i \in N_p^-, \tag{5}$$

and removing the redundant constraints from (TP'). As such, solving (TP(C)) yields a feasible solution of problem (TP'). Note that, compared with the original problem (TP'), problem (TP(C)) has fewer variables and constraints, and particularly, fewer bilinear terms, which makes it much easier to be solved. To further illustrate this, let us consider the special case in which $C = \{\tau\}$ contains a single terminal. In this case, constraints (4g) reduce to $y_{ip} = x^\tau_{ip}$ for $p \in P_\tau$ and $i \in N_p^-$ and constraints (4f) become redundant. Therefore, problem (TP(C)) reduces to an LP problem (that can usually be solved efficiently using state-of-the-art LP solvers).

REMARK 1. Letting $\tau \in T$, and o^* and o^τ be the objective values of problems (TP') and (TP($\{\tau\}$)), respectively, then it follows $o^* \le o^\tau \le 0$.

For any C, we can recover a feasible solution for (TP') from a feasible solution of problem (TP(C)). The following proposition shows that we can derive a feasible solution to (TP(C)) from a feasible solution of the problem (TP') as well.

PROPOSITION 2. *Letting* $(\hat{x},\, \hat{y})$ *be a feasible solution of* (TP') *and* $(\bar{x}^C,\, \bar{y}^C)$ *be defined by*

$$\bar{x}^t_{ip} = \hat{x}^t_{ip},\ \forall\, t \in C,\ p \in P_t,\ i \in N_p^-, \quad \bar{y}_{ip} = \sum_{t\in T_p\cap C} \hat{x}^t_{ip},\ \forall\, p \in P_C,\ i \in N_p^-, \quad \bar{y}_{it} = \hat{y}_{it},\ \forall\, t \in C,\ i \in N_t^-, \tag{6}$$

where $C \subseteq T$. *Then* $(\bar{x}^C,\, \bar{y}^C)$ *is a feasible solution of problem* (TP(C)).

Proof. It suffices to prove that (4f) hold at $(\bar{x}^C,\, \bar{y}^C)$. By Proposition 1, there exists a vector $\hat{q}$ for which $(\hat{x}, \hat{y}, \hat{q})$ is a feasible solution for problem (TP). Letting $t \in C$, $p \in P_t$, $i_1, i_2 \in N_p^-$ with

$i_1 \neq i_2$, for each $t' \in T_p \cap C$, then it follows

$$\bar{x}^t_{i_1p}\bar{x}^{t'}_{i_2p} \overset{\text{(a)}}{=} (\hat{q}^t_p\hat{y}_{i_1p})(\hat{q}^{t'}_p\hat{y}_{i_2p}) = (\hat{q}^{t'}_p\hat{y}_{i_1p})(\hat{q}^t_p\hat{y}_{i_2p}) \overset{\text{(b)}}{=} \bar{x}^{t'}_{i_1p}\bar{x}^t_{i_2p}, \tag{7}$$

where (a) and (b) follow from (6) and (1f) (as $(\hat{x},\ \hat{y},\ \hat{q})$ is feasible to (TP)). Summing constraints (7) over $t' \in T_p \cap C$ and using (6), we obtain $\bar{x}^t_{i_1p}\bar{y}_{i_2p} = \bar{x}^t_{i_2p}\bar{y}_{i_1p}$, which completes the proof. □

3.2. Augmentation problem

Let $(\bar{x}^C, \bar{y}^C)$ be a feasible solution of (TP(C)) (not necessarily optimal) and $\tau \in T\backslash C$. Next, we will discuss how to augment $(\bar{x}^C, \bar{y}^C)$ and obtain a feasible solution of problem (TP($C \cup \{\tau\}$)) with a better objective value (thereby obtaining an improved feasible solution for (TP')). In particular, we solve an augmentation problem, defined by fixing

$$x^t_{ip} = \bar{x}^t_{ip},\ \forall\, t \in C,\ p \in P_t,\ i \in N^-_p, \quad y_{ij} = \bar{y}_{ij},\ \forall\, j \in (C \cup P_C)\backslash P_\tau,\ i \in N^-_j, \tag{8}$$

in problem (TP($C \cup \{\tau\}$)). By removing the fixed variables and redundant constraints from problem (TP($C \cup \{\tau\}$)) with (8), we obtain an equivalent augmentation problem defined as follows:

$$\min_{x^\tau,\, y^\tau} \left\{ \sum_{p\in P_\tau}\sum_{i\in N^-_p} c_{ip}x^\tau_{ip} + \sum_{i\in N^-_\tau} c_{i\tau}y_{i\tau} : (9a)\text{–}(9i) \right\}, \tag{TP($C;\tau$))}$$

where

$$\sum_{t\in C}\left(\sum_{p\in P_t\cap N^+_i}\bar{x}^t_{ip}\right) + \sum_{t\in N^+_i\cap C}\bar{y}_{it} + \sum_{p\in P_\tau\cap N^+_i} x^\tau_{ip} + \chi_{i\tau}y_{i\tau} \leq u_i,\ \forall\, i \in S_\tau \cup P_\tau, \tag{9a}$$

$$\sum_{i\in N^-_\tau} y_{i\tau} \leq u_\tau, \tag{9b}$$

$$\sum_{t\in T_p\cap C}\bar{x}^t_{ip} + x^\tau_{ip} \leq u_{ip},\ \forall\, p \in P_\tau,\ i \in N^-_p, \quad y_{i\tau} \leq u_{i\tau},\ \forall\, i \in N^-_\tau, \tag{9c}$$

$$\sum_{i\in N^-_p} x^\tau_{ip} = \sum_{p'\in P_\tau\cap N^+_p} x^\tau_{pp'} + \chi_{p\tau}y_{p\tau},\ \forall\, p \in P_\tau, \tag{9d}$$

$$\lambda^{\min}_{\tau k}\sum_{i\in N^-_\tau} y_{i\tau} \leq \sum_{s\in S_\tau}\sum_{p\in P_\tau\cap N^+_s}\lambda_{sk}x^\tau_{sp} + \sum_{s\in S_\tau\cap N^-_\tau}\lambda_{sk}y_{s\tau} \leq \lambda^{\max}_{\tau k}\sum_{i\in N^-_\tau} y_{i\tau},\ \forall\, k \in K, \tag{9e}$$

$$\bar{x}^t_{i_1p}y_{i_2p} = \bar{x}^t_{i_2p}y_{i_1p},\ \forall\, t \in C,\ p \in P_t \cap P_\tau,\ i_1, i_2 \in N^-_p \text{ with } i_1 \neq i_2, \tag{9f}$$

$$x^\tau_{i_1p}y_{i_2p} = x^\tau_{i_2p}y_{i_1p},\ \forall\, p \in P_\tau,\ i_1, i_2 \in N^-_p \text{ with } i_1 \neq i_2, \tag{9g}$$

$$y_{ip} = \sum_{t\in T_p\cap C}\bar{x}^t_{ip} + x^\tau_{ip},\ \forall\, p \in P_\tau,\ i \in N^-_p, \tag{9h}$$

$$x^\tau_{ip} \in \mathbb{R}_+,\ \forall\, p \in P_\tau,\ i \in N^-_p,\ y_{ij} \in \mathbb{R}_+,\ \forall\, j \in P_\tau \cup \{\tau\},\ i \in N^-_j. \tag{9i}$$

Solving problem (TP($C;\tau$)), we obtain a feasible solution for problem (TP($C \cup \{\tau\}$)) with an objective value better than that of $(\bar{x}^C, \bar{y}^C)$ for problem (TP(C)).

3.2.1. Transforming the augmentation problem into an LP problem In general, due to the nonlinear constraints (9g), problem (TP($C;\tau$)) is an NLP problem. In the following, we will equivalently transform it into an LP problem. To proceed, observe that as $(\bar{x}^C, \bar{y}^C)$ is a feasible solution of (TP(C)), (4f) and (4g) must hold at $(\bar{x}^C, \bar{y}^C)$. Substituting (4g) into (4f) and evaluating

it at $(\bar{x}^C, \bar{y}^C)$, we obtain $\bar{x}^t_{i_1p} \sum_{t' \in T_p \cap C} \bar{x}^{t'}_{i_2p} = \bar{x}^t_{i_2p} \sum_{t' \in T_p \cap C} \bar{x}^{t'}_{i_1p}$. This, together with (9h), implies that (9f) can be equivalently presented as follows:

$$\bar{x}^t_{i_1p} x^\tau_{i_2p} = \bar{x}^t_{i_2p} x^\tau_{i_1p}, \ \forall\, t \in C, \ p \in P_t \cap P_\tau, \ i_1, i_2 \in N_p^- \text{ with } i_1 \neq i_2. \tag{10}$$

For each $p \in P_\tau$, $i_1, i_2 \in N_p^-$ with $i_1 \neq i_2$, by summing (10) for $t \in T_p \cap C$, we obtain $x^\tau_{i_1p}(\sum_{t \in T_p \cap C} \bar{x}^t_{i_2p}) = x^\tau_{i_2p}(\sum_{t \in T_p \cap C} \bar{x}^t_{i_1p})$, which, together with (9h), implies $x^\tau_{i_1p}(y_{i_2p} - x^\tau_{i_2p}) = x^\tau_{i_2p}(y_{i_1p} - x^\tau_{i_1p})$, or equivalently, $x^\tau_{i_1p} y_{i_2p} = x^\tau_{i_2p} y_{i_1p}$. As a result, (9g) can be equivalently removed from problem (TP($C;\tau$)). Substituting (9f) with (10) and removing (9g), we obtain the following LP problem:

$$\min_{x^\tau, y^\tau} \left\{ \sum_{p \in P_\tau} \sum_{i \in N_p^-} c_{ip} x^\tau_{ip} + \sum_{i \in N_\tau^-} c_{i\tau} y_{i\tau} : \text{(9a)–(9e)}, \ \text{(9h)}, \ \text{(9i)}, \ \text{(10)} \right\}. \tag{TP$_\text{L}(C;\tau)$}$$

THEOREM 1. *Problems* (TP($C;\tau$)) *and* (TP$_\text{L}(C;\tau)$) *are equivalent.*

Theorem 1 implies that to augment an existing feasible solution of (TP(C)) and obtain a better solution for (TP($C \cup \{\tau\}$)), one can solve the LP augmentation problem (TP$_\text{L}(C;\tau)$), which is easier to solve than the NLP augmentation problem (TP($C;\tau$)).

3.2.2. Analyzing the relations among the augmentation problems and closing the open question in Gupte et al. (2017) Note that given a feasible solution (x^C, y^C) of (TP(C)), we can iteratively augment it until $C = T$, in order to obtain a high-quality solution for problem (TP'). The following proposition shows that, given a fixed $\tau \in T$, the optimal value of augmentation problem (TP$_\text{L}(C;\tau)$) is monotonically nonincreasing with respect to the subset C.

PROPOSITION 3. *Given two subsets $C, C' \subseteq T$ with $C \subseteq C'$ and $\tau \in T \backslash C'$, denote $(\hat{x}^{C'}, \hat{y}^{C'})$ as the feasible solution of* (TP(C')) *(iteratively) augmented from the feasible solution $(\bar{x}^C, \bar{y}^C)$ of* (TP(C)), *that is,*

$$\bar{x}^t_{ip} = \hat{x}^t_{ip}, \ \forall\, t \in C, \ p \in P_t, \ i \in N_p^-, \quad \bar{y}_{it} = \hat{y}_{it}, \ \forall\, t \in C, \ i \in N_t^-. \tag{11}$$

Letting o and o' be the optimal values of problems (TP$_\text{L}(C;\tau)$) *and* (TP$_\text{L}(C';\tau)$) *defined by $(\bar{x}^C, \bar{y}^C)$ and $(\hat{x}^{C'}, \hat{y}^{C'})$, respectively, then $o \leq o'$. In particular, if $o = 0$, then $o' = 0$.*

Proof. Letting $(\check{x}^\tau, \check{y}^\tau)$ be the optimal solution of the augmentation problem (TP$_\text{L}(C';\tau)$), and $(\tilde{x}^\tau, \tilde{y}^\tau)$ be defined as $\tilde{x}^\tau = \check{x}^\tau$ and

$$\tilde{y}_{i\tau} = \check{y}_{i\tau}, \ \forall\, i \in N_\tau^-, \quad \tilde{y}_{ip} = \sum_{t \in T_p \cap C} \bar{x}^t_{ip} + \check{x}^\tau_{ip}, \ \forall\, p \in P_\tau, \ i \in N_p^-.$$

To prove the first part, it suffices to prove that $(\tilde{x}^\tau, \tilde{y}^\tau)$ is a feasible solution of problem (TP$_\text{L}(C;\tau)$). By the definition of $(\tilde{x}^\tau, \tilde{y}^\tau)$, $C \subseteq C'$, and the feasibility of $(\check{x}^\tau, \check{y}^\tau)$ for (TP$_\text{L}(C';\tau)$), constraints (9b), (9d), (9e), (9h), (9i), and (10) of problem (TP$_\text{L}(C;\tau)$) hold at $(\tilde{x}^\tau, \tilde{y}^\tau)$. From (11), the remaining constraints (9a) and (9c) in problem (TP$_\text{L}(C;\tau)$) are implied by the corresponding constraints in problem (TP$_\text{L}(C';\tau)$), and thus also hold at $(\tilde{x}^\tau, \tilde{y}^\tau)$. The second part follows from the fact that (x^τ, y^τ) defined by $x^\tau = \mathbf{0}$, $y_{i\tau} = 0$ for $i \in N_\tau^-$, and $y_{ip} = \sum_{t \in T_p \cap C} \bar{x}^t_{ip}$ for $p \in P_\tau$ and $i \in N_p^-$ is a feasible solution of (TP$_\text{L}(C';\tau)$) with an objective value of 0. □

It is known that the trivial solution $(x, y) = (\mathbf{0}, \mathbf{0})$ is a feasible solution of the pooling problem (TP') with an objective value of zero; however, whether checking the triviality of a pooling problem instance—determining whether the optimal value is zero—can be achieved in polynomial time is still an open question (Gupte et al. 2017, Remark 2.2). To close this open question, we first use Proposition 3 to derive a lower bound for problem (TP') (by solving the LP subproblems (TP($\{t\}$))).

PROPOSITION 4. *Let o^* and o^t be the optimal values of* (TP') *and* (TP($\{t\}$)) *(where $t \in T$). Then $o^* \geq \sum_{t\in T} o^t$.*

Proof. Letting $(\hat{x},\ \hat{y})$ be an optimal solution of (TP'), then $o^* = \sum_{t\in T}(\sum_{p\in P_t}\sum_{i\in N_p^-} c_{ip}\hat{x}_{ip}^t + \sum_{i\in N_t^-} c_{it}\hat{y}_{it})$. It suffices to prove

$$\hat{o}^t := \sum_{p\in P_t}\sum_{i\in N_p^-} c_{ip}\hat{x}_{ip}^t + \sum_{i\in N_t^-} c_{it}\hat{y}_{it} \geq o^t,\ \forall\, t \in T. \tag{12}$$

Let $\tau \in T$ and consider the subproblem (TP$_{\rm L}(T\backslash\{\tau\};\{\tau\})$) defined by $(\bar{x}^{T\backslash\{\tau\}}, \bar{y}^{T\backslash\{\tau\}})$ in (6) with $C = T\backslash\{\tau\}$. Then, the point $(\tilde{x}^\tau, \tilde{y}^\tau)$ defined by

$$\tilde{x}_{ip}^\tau = \hat{x}_{ip}^\tau,\ \forall\, p \in P_\tau,\ i \in N_p^-, \quad \tilde{y}_{i\tau} = \hat{y}_{i\tau},\ \forall\, i \in N_\tau^-, \quad \tilde{y}_{ip} = \sum_{t\in T_p} \hat{x}_{ip}^t,\ \forall\, p \in P_\tau,\ i \in N_p^-,$$

must also be an optimal solution of (TP$_{\rm L}(T\backslash\{\tau\};\{\tau\})$) with an objective value $\hat{o}^\tau$ (otherwise, a better solution than $(\hat{x},\hat{y})$ can be obtained, a contradiction with the optimality of $(\hat{x},\hat{y})$ for (TP')). Letting $C = \varnothing$ and $C' = T\backslash\{\tau\}$, then by Proposition 3, it follows $\hat{o}^\tau \geq o^\tau$, which completes the proof. □

Based on Remark 1 and Proposition 4, and the fact that (TP(C)) with $C = \{\tau\}$ is an LP problem, we can provide a positive answer to the open question of Gupte et al. (2017).

THEOREM 2. *Let o^* be the optimal value of problem* (TP'). *Then checking whether $o^* = 0$ can be done in polynomial time.*

Proof. To evaluate whether $o^* = 0$, we first solve the single terminal problem (TP($\{t\}$)) for all $t \in T$ to obtain optimal values $\{o^t\}_{t\in T}$, and then consider the following two cases:

(i) If $o^{t_0} < 0$ holds for some $t_0 \in T$, then from Remark 1, we obtain $o^* \leq o^{t_0} < 0$.

(ii) Otherwise, from Proposition 4, $o^* \geq \sum_{t\in T} o^t \geq 0$ holds, which, together with $o^* \leq 0$ (from Remark 1), implies that $o^* = 0$.

Since the LP problem (TP($\{t\}$)) can be solved in polynomial time, checking whether $o^* = 0$ can be accomplished in polynomial time. □

3.3. The dive-and-fix framework

Based on the results established in Sections 3.1 and 3.2, we can design a heuristic algorithm that starts with $C := \varnothing$ and the trivial solution $(\mathbf{0},\mathbf{0})$ and iteratively augments the incumbent solution by solving problem (TP$_{\rm L}(C;\tau)$) corresponding to terminal $\tau \in T\backslash C$ until all candidate terminals are considered, for solving the pooling problem (TP'). Specifically, letting (x^C, y^C) be the incumbent solution encountered in an iteration where $C \subseteq T$, then we attempt to find a terminal that leads to the best objective improvement: $\tau_{\min} \in \arg\min_{\tau\in T\backslash C} \hat{o}^\tau$, where $\hat{o}^\tau$ is the optimal value of the augmentation problem (TP$_{\rm L}(C;\tau)$) for $\tau \in T\backslash C$. Subsequently, we construct an improved feasible solution $(\tilde{x}^{C\cup\{\tau_{\min}\}}, \tilde{y}^{C\cup\{\tau_{\min}\}})$ (feasible for problem (TP($C\cup\{\tau_{\min}\}$))):

$$\begin{aligned}
&\tilde{x}_{ip}^t = \bar{x}_{ip}^t,\ \forall\, t \in C,\ p \in P_t,\ i \in N_p^-, \quad \tilde{x}_{ip}^{\tau_{\min}} = \hat{x}_{ip}^{\tau_{\min}},\ \forall\, p \in P_{\tau_{\min}},\ i \in N_p^-,\\
&\tilde{y}_{ip} = \sum_{t\in T_p\cap(C\cup\{\tau_{\min}\})} \tilde{x}_{ip}^t,\ \forall\, p \in P_{C\cup\{\tau_{\min}\}},\ i \in N_p^-, \quad \tilde{y}_{it} = \bar{y}_{it},\ \forall\, t \in C,\ i \in N_t^-, \quad \tilde{y}_{i\tau_{\min}} = \hat{y}_{i\tau_{\min}},\ \forall\, i \in N_{\tau_{\min}}^-,
\end{aligned} \tag{13}$$

where $(\hat{x}^{\tau_{\min}}, \hat{y}^{\tau_{\min}})$ is an optimal solution of problem (TP$_{\rm L}(C;\tau_{\min})$), and update $(\bar{x}^C, \bar{y}^C) \leftarrow (\tilde{x}^{C\cup\{\tau_{\min}\}}, \tilde{y}^{C\cup\{\tau_{\min}\}})$ and $C \leftarrow C \cup \{\tau_{\min}\}$. The procedure is repeated until $C = T$.

Algorithm 1 summarizes the above dive-and-fix heuristic algorithm for the pooling problem (TP'). Note that in step 6, if $\hat{o}^\tau = 0$, then by Proposition 3, the optimal value of the augmentation problems corresponding to terminal τ in the future iterations will be 0. This implies that augmenting

Algorithm 1: A dive-and-fix heuristic method for pooling problem

Let $C \leftarrow \emptyset$ and $\tilde{T} \leftarrow T \backslash C$;
while $\tilde{T} \neq \emptyset$ **do**
  Set $\tau_{\min} = -1$ and $\hat{o}^{\tau_{\min}} = 0$;
  for $\tau \in \tilde{T}$ **do**
    Solve $(\mathrm{TP_L}(C;\tau))$ to obtain an optimal solution $(\hat{x}^\tau, \hat{y}^\tau)$ with the objective value $\hat{o}^\tau$;
    If $\hat{o}^\tau = 0$, set $\tilde{T} \leftarrow \tilde{T} \backslash \{\tau\}$;
    If $\hat{o}^\tau < \hat{o}^{\tau_{\min}}$, update $\tau_{\min} \leftarrow \tau$ and $\hat{o}^{\tau_{\min}} \leftarrow \hat{o}^\tau$;
  If $\hat{o}^{\tau_{\min}} < 0$, set $(\bar{x}^C, \bar{y}^C) \leftarrow (\bar{x}^{C\cup\{\tau_{\min}\}}, \bar{y}^{C\cup\{\tau_{\min}\}})$ (defined in (13))), $C \leftarrow C \cup \{\tau_{\min}\}$, $\tilde{T} \leftarrow \tilde{T} \backslash \{\tau_{\min}\}$; otherwise **stop** with the feasible solution $(\bar{x}^C, \bar{y}^C)$;

the solution by considering terminal τ cannot lead to a better feasible solution, and therefore we omit it in the future iterations.

It is worthy remarking that Alfaki and Haugland (2014) also developed a heuristic algorithm that iteratively augments the current solution (that considers only a subset of terminals) by incorporating a new terminal. However, the approach of Alfaki and Haugland (2014) is based on the P/PQ formulation (Haverly 1978, Tawarmalani and Sahinidis 2002) and in each iteration, an NLP augmentation subproblem is solved, which could be time-consuming. In contrast, the proposed dive-and-fixing algorithm is based on the TP formulation, which, compared with the one in Alfaki and Haugland (2014), can still guarantee to return a feasible solution for problem (TP') while requiring solving LP problems of the form $(\mathrm{TP_L}(C;\tau))$ in each iteration, thereby making it run very efficiently in practice; see the computational results in the next section.

4. Computational results

In this section, we present computational results to demonstrate the effectiveness and efficiency of the proposed dive-and-fix algorithm (denoted as `D&F`). To do this, we compare the proposed dive-and-fix algorithm with the global branch-and-bound algorithm based on formulation (TP) (denoted as `B&B`) and the greedy construction heuristic algorithm from Alfaki and Haugland (2014) (denoted as `A&H`). Following Alfaki and Haugland (2014), we first solve the single terminal problems (which are LP problems) and then iteratively augment the current solution according to the increasing optimal values of single terminal problems, to avoid solving too many (time-consuming) NLP augmentation problems. All three algorithms were implemented in Julia 1.10.6. The NLP problems arising in `B&B` and `A&H` were solved by Baron 24.12.21 and the LP problems arising in `A&H` and `D&F` were solved by CPLEX 20.1.0. We set a time limit of 7200 seconds for `B&B` and, following Alfaki and Haugland (2014), a time limit of $7200/|T|$ for each NLP augmentation problem of `A&H`. All experiments were run on a cluster equipped with Intel(R) Xeon(R) Gold 6140 CPU @ 2.30GHz computers.

We first compare the performance of settings `D&F`, `A&H`, and `B&B` on a testbed of 34 benchmark standard pooling problem instances where $A \cap (P \times P) = \emptyset$; see Alfaki and Haugland (2013b, Section 7.5) for a detailed description of these instances. Table 1 summarizes the performance results of settings `B&B`, `A&H`, and `D&F`. We report for each instance the instance id and the numbers of sources, pools, terminals, and quality attributes (`id-S-P-T-K`), the CPU time in seconds (`T`), the optimal value of the returned feasible solution under each setting (`Obj`), and the optimality gap of the objective values (`G(%)`). The optimality gap `G(%)` is defined as $|\frac{\mathtt{LB}-o}{\mathtt{LB}}| \times 100\%$, where `LB` denotes the optimal value or the best lower bound (returned by `B&B`) and $o \in \{o_{\mathtt{B\&B}}, o_{\mathtt{A\&H}}, o_{\mathtt{D\&F}}\}$

Table 1 Comparison of B&B, A&H, and D&F on the standard pooling problem instances.

id-S-P-T-K	LB	B&B			A&H			D&F			
		T	Obj	G(%)	T	Obj	G(%)	T	Obj	G(%)	#SAP
Adhya1-5-2-4-4	-549.8	3.3	-549.8	0.0	0.2	-541.5	1.5	<0.1	-509.8	7.3	6
Adhya2-5-2-4-6	-549.8	3.6	-549.8	0.0	0.2	-541.5	1.5	<0.1	-509.8	7.3	5
Adhya3-8-3-4-6	-561.0	2.4	-561.0	0.0	0.2	-552.8	1.5	<0.1	-552.8	1.5	5
Adhya4-8-2-5-4	-877.6	1.8	-877.6	0.0	0.2	-877.6	0.0	<0.1	-877.6	0.0	9
Bental4-4-1-2-1	-450.0	0.3	-450.0	0.0	<0.1	-450.0	0.0	<0.1	-450.0	0.0	3
Bental5-5-3-5-2	-3500.0	3.3	-3500.0	0.0	0.2	-3500.0	0.0	<0.1	-3500.0	0.0	15
Foulds2-6-2-4-1	-1100.0	0.2	-1100.0	0.0	0.1	-1100.0	0.0	<0.1	-1000.0	9.1	9
Foulds3-11-8-16-1	-8.0	2.4	-8.0	0.0	0.7	-8.0	0.0	0.1	-8.0	0.0	136
Foulds4-11-8-16-1	-8.0	5.6	-8.0	0.0	0.7	-8.0	0.0	0.1	-8.0	0.0	136
Foulds5-11-4-16-1	-8.0	47.1	-8.0	0.0	1.9	-7.4	6.9	0.3	-8.0	0.0	136
Haverly1-3-1-2-1	-400.0	0.3	-400.0	0.0	<0.1	-400.0	0.0	<0.1	-400.0	0.0	3
Haverly2-3-1-2-1	-600.0	0.5	-600.0	0.0	<0.1	-600.0	0.0	<0.1	-600.0	0.0	3
Haverly3-3-1-2-1	-750.0	0.6	-750.0	0.0	<0.1	-750.0	0.0	<0.1	-750.0	0.0	3
RT2-3-2-3-8	-6697.7	<0.1	-6697.7	0.0	<0.1	-6697.7	0.0	<0.1	-6697.7	0.0	5
sppA0-20-10-15-24	-37210.4	7200.0	-29480.6	20.8	1.0	-16163.5	56.6	<0.1	-24773.1	33.4	42
sppA1-20-10-15-24	-30201.0	7200.0	-29207.4	3.3	1.5	-22818.0	24.4	<0.1	-23084.9	23.6	43
sppA2-20-10-15-24	-23459.0	7200.0	-22828.7	2.7	962.0	-13895.3	40.8	<0.1	-16010.3	31.8	30
sppA3-20-10-15-24	-41695.3	7200.0	-34868.9	16.4	1.8	-26804.9	35.7	0.1	-27688.2	33.6	53
sppA4-20-10-15-24	-43096.7	7200.0	-13297.2	69.1	2.1	-28886.3	33.0	0.2	-30688.0	28.8	57
sppA5-20-10-15-24	-28257.8	7200.0	-17437.8	38.3	13.7	-18593.2	34.2	0.2	-18360.1	35.0	43
sppA6-20-10-15-24	-42463.0	7200.0	-38682.6	8.9	2.6	-28202.2	33.6	0.3	-27332.9	35.6	63
sppA7-20-10-15-24	-44682.2	7200.0	-36797.1	17.6	10.6	-31231.4	30.1	0.4	-32380.5	27.5	45
sppA8-20-10-15-24	-30666.9	7200.0	-29279.9	4.5	33.7	-24406.5	20.4	0.6	-24322.5	20.7	71
sppA9-20-10-15-24	-21934.0	7200.0	-19511.4	11.0	555.2	-16217.1	26.1	0.3	-14342.8	34.6	33
sppB0-35-17-21-34	-45395.1	7200.0	-24950.0	45.0	33.8	-29256.5	35.6	0.3	-26959.4	40.6	77
sppB1-35-17-21-34	-65258.9	7200.0	-17711.9	72.9	6.0	-45605.9	30.1	0.7	-48555.6	25.6	99
sppB2-35-17-21-34	-56515.2	7200.0	-19836.4	64.9	2831.0	-36982.3	34.6	1.9	-39367.9	30.3	113
sppB3-35-17-21-34	-74050.5	7200.0	-32886.3	55.6	246.0	-54768.2	26.0	3.0	-50596.5	31.7	110
sppB4-35-17-21-34	-59469.7	7200.0	-21095.1	64.5	2586.4	-49521.9	16.7	6.8	-48532.8	18.4	112
sppB5-35-17-21-34	-60696.4	7200.0	-16255.1	73.2	1258.4	-44909.7	26.0	9.9	-45548.0	25.0	124
sppC0-60-30-40-40	-98398.1	7200.0	-1419.6	98.6	8.2	-48930.5	50.3	1.2	-59095.7	39.9	320
sppC1-60-30-40-40	-120312.0	7200.0	-41816.8	65.2	12.5	-54915.6	54.4	2.1	-66350.9	44.9	256
sppC2-60-30-40-40	-136397.0	7200.0	-966.1	99.3	60.3	-64222.4	52.9	4.6	-77124.1	43.5	319
sppC3-60-30-40-40	-130315.0	7200.0	0.0	100.0	989.5	-81788.4	37.2	7.2	-77935.5	40.2	303
Average		4237.4		27.4	283.0		20.9	1.2		19.7	

denotes the objective value returned by the corresponding settings. For the proposed dive-and-fix algorithm, we also report the number of solved augmentation problems ($\mathrm{TP_L}(C;\tau)$) (#SAP).

From Table 1, we observe that B&B can only solve small-scale instances (i.e., Adhya1-5-2-2-4 to RT2-3-2-3-8) to optimality within the time limit of 7200 seconds. For the unsolved instances, the solution quality of the feasible solution returned by B&B is generally poor, usually much worse than that of the solutions returned by A&H and D&F; see columns G(%) . Compared with the greedy algorithm of Alfaki and Haugland (2014), the proposed dive-and-fix algorithm can find slightly better solutions and has a significantly better computational efficiency. In particular, the average optimality gap and CPU time returned by D&F are 19.7% and 1.2 seconds, respectively, while those returned by A&H are 20.9% and 283.0 seconds, respectively. The computational efficiency of D&F over A&H is mainly attributed to the fact that D&F only requires solving LP augmentation problems of the form ($\mathrm{TP_L}(C;\tau)$), whereas A&H requires solving (time-consuming) NLP augmentation problems. In addition, the number of solved LP augmentation problems in D&F are not large (as shown in column #SAP), which demonstrates the advantage of Proposition 3, i.e., it avoids solving LP augmentation problem ($\mathrm{TP_L}(C;\tau)$) that cannot lead to a better solution.

Next, we present the computational results for the generalized pooling problem (GPP). We follow Alfaki and Haugland (2013a) to construct a testbed of GPP instances, where the numbers of source, pool, terminal, and attribute qualities are chosen as in Table 2. Each possible arc $(i,j) \in (S \times P) \cup (P \times P) \cup (S \times T)$ is included with a probability of 50%, while the arcs in $P \times T$

Table 2 Comparison of B&B, A&H, and D&F on the generalized pooling problem instances.

id-S-P-T-K	LB	B&B			A&H			D&F			
		T	Obj	G(%)	T	Obj	G(%)	T	Obj	G(%)	#SAP
F1-30-10-20-8	-4340.2	7200.0	-3332.1	23.2	10.8	-2281.0	47.4	2.1	-3155.2	27.3	67
F2-30-10-20-8	-2861.0	7200.0	-2668.3	6.7	1089.8	-1930.7	32.5	0.9	-1843.3	35.6	36
F3-30-10-20-8	-2915.4	7200.0	-2077.3	28.7	36.5	-1958.3	32.8	1.4	-2105.6	27.8	54
F4-30-10-20-8	-2687.2	7200.0	-2563.9	4.6	1819.0	-2492.5	7.2	1.1	-2459.1	8.5	37
F5-30-10-20-8	-4598.5	7200.0	-4137.6	10.0	16.7	-3280.5	28.7	1.9	-3554.9	22.7	74
G1-40-11-30-9	-4219.3	7200.0	-2737.0	35.1	991.9	-2776.7	34.2	4.1	-2707.0	35.8	99
G2-40-11-30-9	-6835.1	7200.0	-5037.6	26.3	1954.7	-4592.1	32.8	11.1	-4843.5	29.1	202
G3-40-11-30-9	-7260.3	7200.0	-3529.5	51.4	748.7	-2596.7	64.2	5.0	-2992.9	58.8	97
G4-40-11-30-9	-6075.9	7200.0	-3253.0	46.5	753.8	-2041.0	66.4	6.6	-3725.3	38.7	140
G5-40-11-30-9	-3097.8	7200.0	-2766.3	10.7	25.2	-1438.0	53.6	1.9	-2501.7	19.2	53
H1-50-12-40-11	-5874.1	7200.0	-2371.8	59.6	606.9	-2096.4	64.3	6.7	-4307.6	26.7	100
H2-50-12-40-11	-3943.1	7200.0	-2190.4	44.4	69.7	-960.0	75.7	14.7	-2584.8	34.4	163
H3-50-12-40-11	-3917.4	7200.0	-1886.7	51.8	275.4	-2588.8	33.9	4.3	-2435.1	37.8	76
H4-50-12-40-11	-5679.9	7200.0	-2506.3	55.9	427.7	-1407.6	75.2	6.9	-2320.5	59.1	90
H5-50-12-40-11	-7068.9	7200.0	-2955.2	58.2	60.9	-1594.0	77.5	10.9	-3801.6	46.2	132
I1-60-13-50-14	-5398.8	7200.0	0.0	100.0	158.9	-1502.0	72.2	24.7	-3308.4	38.7	176
I2-60-13-50-14	-7973.8	7200.0	-3669.6	54.0	215.0	-2292.0	71.3	32.9	-4759.1	40.3	200
I3-60-13-50-14	-3643.1	7200.0	0.0	100.0	152.6	-984.0	73.0	10.1	-1734.7	52.4	106
I4-60-13-50-14	-3676.7	7200.0	-480.9	86.9	742.3	-3199.1	13.0	8.3	-2999.4	18.4	90
I5-60-13-50-14	-2823.9	7200.0	0.0	100.0	798.5	-2399.2	15.0	4.3	-2188.7	22.5	68
J1-70-14-60-17	-2291.0	7200.0	0.0	100.0	279.7	-1877.0	18.1	5.9	-1755.3	23.4	74
J2-70-14-60-17	-2458.3	7200.0	0.0	100.0	174.9	-1098.0	55.3	7.0	-1446.4	41.2	78
J3-70-14-60-17	-3835.6	7200.0	-1237.5	67.7	558.9	-2166.6	43.5	6.7	-2650.4	30.9	79
J4-70-14-60-17	-4025.8	7200.0	0.0	100.0	550.1	-2145.0	46.7	8.7	-1895.9	52.9	87
J5-70-14-60-17	-1428.6	7200.0	-404.4	71.7	628.5	-1293.7	9.4	5.2	-1259.4	11.8	70
Average		7200.0		55.7	525.9		45.8	7.7		33.6	

are fully preserved. Each arc has a weight of $c_{ij} = c_j - c_i$, where $\{c_s\}_{s\in S}$ and $\{c_t\}_{t\in T}$ are uniformly chosen from $\{0,\ldots,5\}$, and $\{5,\ldots,14\}$, respectively, and $\{c_p\}_{p\in P}$ are set to zero. The node capacities $\{u_i\}_{i\in S\cup P\cup T}$ are uniformly chosen from $\{20,\ldots,59\}$, while the arc capacities $\{u_{ij}\}_{(i,j)\in A}$ are left unbounded. The quality attributes $\{\lambda_{sk}\}_{s\in S,\,k\in K}$ at the sources are drawn randomly from $\{0,\ldots,9\}$; the lower and upper bounds for the quality attributes at the terminals $\{\lambda^{\min}_{tk}\}_{t\in T,\,k\in K}$ and $\{\lambda^{\max}_{tk}\}_{t\in T,\,k\in K}$ are fixed at 0 and chosen from $\{2,\ldots,6\}$, respectively.

Table 2 summarizes the numerical results for the GPP instances. Similarly, we can observe that B&B is generally outperformed by A&H and D&F in terms of solution quality and computational efficiency; indeed, B&B fails to solve all instances to optimality within the time limit of 7200 seconds, and for some instances, it can only return the all zeros trivial solution. Compared with A&H, the proposed D&F is not only significantly more computationally efficient, but also finds fairly better feasible solutions. The latter is attributed to D&F's strategy in selecting the terminal for augmenting the current solution: it selects the terminal that provides the greatest objective improvement in each iteration, rather than following a predetermined ordering as A&H does (which is due to the computational overhead associated with solving the NLP augmentation problems).

In summary, our computational results illustrate the effectiveness and efficiency of the proposed dive-and-fix algorithm for solving pooling problems. In particular, compared with the B&B algorithm and the greedy construction heuristic algorithm of Alfaki and Haugland (2014), our proposed dive-and-fix algorithm can generally find a better solution and is significantly more computationally efficient.

References

Alfaki M, Haugland D (2013a) A multi-commodity flow formulation for the generalized pooling problem. *J. Global Optim.* 56(3):917–937.

Alfaki M, Haugland D (2013b) Strong formulations for the pooling problem. *J. Global Optim.* 56(3):897–916.

Alfaki M, Haugland D (2014) A cost minimization heuristic for the pooling problem. *Ann. Oper. Res.* 222(1):73–87.

Audet C, Brimberg J, Hansen P, Digabel SL, Mladenović N (2004) Pooling problem: Alternate formulations and solution methods. *Manag. Sci.* 50(6):761–776.

Baker TE, Lasdon LS (1985) Successive linear programming at Exxon. *Manag. Sci.* 31(3):264–274.

Ben-Tal A, Eiger G, Gershovitz V (1994) Global minimization by reducing the duality gap. *Math. Program.* 63(1-3):193–212.

Blom ML, Burt CN, Pearce AR, Stuckey PJ (2014) A decomposition-based heuristic for collaborative scheduling in a network of open-pit mines. *INFORMS J. Comput.* 26(4):658–676.

Boland N, Kalinowski T, Rigterink F (2016) New multi-commodity flow formulations for the pooling problem. *J. Global Optim.* 66(4):669–710.

Boland N, Kalinowski T, Rigterink F (2017) A polynomially solvable case of the pooling problem. *J. Global Optim.* 67(3):621–630.

Castro PM (2023) Global optimization of QCPs using MIP relaxations with a base-2 logarithmic partitioning scheme. *Ind. Eng. Chem. Res.* 62(28):11053–11066.

Dey SS, Gupte A (2015) Analysis of MILP techniques for the pooling problem. *Oper. Res.* 63(2):412–427.

Floudas CA, Aggarwal A (1990) A decomposition strategy for global optimum search in the pooling problem. *ORSA J. Comput.* 2(3):225–235.

Galan B, Grossmann IE (1998) Optimal design of distributed wastewater treatment networks. *Ind. Eng. Chem. Res.* 37(10):4036–4048.

Grothey A, McKinnon K (2023) On the effectiveness of sequential linear programming for the pooling problem. *Ann. Oper. Res.* 322(2):691–711.

Gupte A, Ahmed S, Dey SS, Cheon MS (2017) Relaxations and discretizations for the pooling problem. *J. Global Optim.* 67(3):631–669.

Haverly CA (1978) Studies of the behavior of recursion for the pooling problem. *ACM SIGMAP Bull.* (25):19–28.

Lasdon L, Joffe B (1990) The relationship between distributive recursion and successive linear programming in refining production planning models. *NPRA Computer Conference* (Seattle, Washington).

Luedtke J, D'Ambrosio C, Linderoth J, Schweiger J (2020) Strong convex nonlinear relaxations of the pooling problem. *SIAM J. Optim.* 30(2):1582–1609.

Meyer CA, Floudas CA (2006) Global optimization of a combinatorially complex generalized pooling problem. *AIChE J.* 52(3):1027–1037.

Misener R, Floudas CA (2009) Advances for the pooling problem: Modeling, global optimization, and computational studies. *Appl. Comput. Math.* 8:3–22.

Misener R, Floudas CA (2010) Global optimization of large-scale generalized pooling problems: Quadratically constrained MINLP models. *Ind. Eng. Chem. Res.* 49(11):5424–5438.

Misener R, Thompson JP, Floudas CA (2011) APOGEE: Global optimization of standard, generalized, and extended pooling problems via linear and logarithmic partitioning schemes. *Comput. Chem. Eng.* 35(5):876–892.

Pham V, Laird C, El-Halwagi M (2009) Convex hull discretization approach to the global optimization of pooling problems. *Ind. Eng. Chem. Res.* 48(4):1973–1979.

Ríos-Mercado RZ, Borraz-Sánchez C (2015) Optimization problems in natural gas transportation systems: A state-of-the-art review. *Appl. Energy* 147:536–555.

Rømo F, Tomasgard A, Hellemo L, Fodstad M, Eidesen BH, Pedersen B (2009) Optimizing the Norwegian natural gas production and transport. *Interfaces* 39(1):46–56.

Tawarmalani M, Sahinidis NV (2002) *Convexification and Global Optimization in Continuous and Mixed-Integer Nonlinear Programming*, volume 65 of *Nonconvex Optimization and Its Applications* (Boston, MA: Springer US).